

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

DISHA D S (1BM23CS094)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **StudentName (1BM23CS000)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Lab incharge Mayanka Gupta Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	11/8/25	Genetic algorithm for optimization - TSP	4
2	25/8/25	Gene expression algorithm – Financial modelling and forecasting	9
3	1/9/25	Particle swarm optimization – Robotics path planning	14
4	8/9/25	Ant colony optimization – Job shop scheduling for manufacturing	20
5	15/9/25	Cuckoo Search - TSP	31
6	29/9/25	Grey wolf optimizer – Image segmentation	38
7	13/10/25	Parallel cellular algorithm – Traffic flow simulation	43

Github Link:

https://github.com/DishaDS094/DISHADS_1BM23CS094_BIS_LAB

PROGRAM 1

Genetic algorithm for optimization – TSP

1. Genetic Algorithm for optimization of travelling salesman problem

Lab-1
Genetic Algorithm for Optimization Problems
11/8/25

Genetic Algorithm is an optimization technique inspired by natural selection.

It works by evolving a population of candidate solutions to improve their quality over time.

We first define the problem & the fitness function that measures how good each solution is.

Initial population is generated randomly.

In each generation, selection chooses the best individual to reproduce & crossover mixes of two solutions to create new ones.

Mutation introduces small random changes to maintain diversity & avoid getting stuck.

This process repeats for a set of generations until best solution stabilizes.

Applications:

- * It is widely used when the search space is large, complex or non-linear.
- * Scheduling, route optimization, engineering design, game AI, machine learning parameter tuning.

Genetic Algorithm Application - Travelling Salesman Problem (TSP)

The travelling salesman problem is one of the most famous optimization problems. A salesman must visit a given set of cities exactly once & return to the starting city while minimizing the total travel distance.

- * TSP is NP-hard - no efficient exact algorithm exists for large no. of cities.
- * Brute force would require checking all possible permutations of cities (which is factorial in complexity).
- * Genetic Algorithm (GA) provides an effective way to find a near optimal solution in reasonable time.
- * Widely used in logistics (Amazon, FedEx), airline scheduling.

→ Why GA is best here:

Other algorithms (like dynamic programming, branch & bound) become impractical when cities are large (50, 100, 1K+). GA scales well & gives close to optimal solutions quickly.

Pseudocode:

Input: Set of cities & distance matrix

1. Initialize population:
 - Generate random routes (permutation of cities)
2. Evaluate fitness of each route:
 - $$Fitness = \frac{1}{total\ distance\ of\ route}$$
3. Repeat until max generations:
 - a. Selection:
 - * Select parents based on fitness

- b. crossover :
 - Apply order crossover (OX) to generate children routes
- c. Mutation:
 - Swap two cities in the route randomly.
- d. Evaluation:
 - Compute distance & fitness for new routes
- e. Replacement:
 - Form new population (elitism keeps best route)
- 4. Return the best route & its total distance

Output:

Number of cities: 10

City Coordinates: [(16, 11), (65, 96), (30, 42), (82, 100), (40, 67), (30, 25), (67, 7), (24, 64), (49, 56), (16, 12)]

Initialization cities & population ...

Starting Genetic Algorithm ...

Generation 1: Best Distance = 305.39

Generation 2: Best Distance = 305.39

Generation 3: Best Distance = 279.97

Generation 4: Best Distance = 279.97

Generation 5: Best Distance = 279.97

Generation 6: Best Distance = 279.97

No improvement for 3 generations. Stopping early

Best Route Found: [9, 0, 6, 5, 7, 1, 3, 8, 4, 2]

Best Distance: 279.97

Python code:

```
import random
```

```
import math
```

```
# Calculate total distance of a route
```

```
def route_distance(route, cities):
```

```
    dist = 0
```

```
    for i in range(len(route)):
```

```
        x1, y1 = cities[route[i]]
```

```
        x2, y2 = cities[route[(i + 1) % len(route)]]
```

```
        dist += math.sqrt((x1 - x2)**2 + (y1 - y2)**2)
```

```
    return dist
```

```
# Create initial population
```

```
def initial_population(pop_size, city_count):
```

```
    population = []
```

```
    for _ in range(pop_size):
```

```
        route = list(range(city_count))
```

```

        random.shuffle(route)
        population.append(route)
    return population

# Tournament selection
def selection(population, cities):
    selected = random.sample(population, 3)
    selected.sort(key=lambda r: route_distance(r, cities))
    return selected[0]

# Order crossover
def crossover(parent1, parent2):
    start, end = sorted(random.sample(range(len(parent1)), 2))
    child = [None] * len(parent1)
    child[start:end] = parent1[start:end]
    pointer = 0
    for gene in parent2:
        if gene not in child:
            while child[pointer] is not None:
                pointer += 1
            child[pointer] = gene
    return child

# Mutation (swap two cities)
def mutate(route, mutation_rate=0.1):
    if random.random() < mutation_rate:
        i, j = random.sample(range(len(route)), 2)
        route[i], route[j] = route[j], route[i]
    return route

# Genetic Algorithm with early stopping
def genetic_algorithm(cities, pop_size=50, generations=100):
    print("Initializing cities and population...")
    population = initial_population(pop_size, len(cities))

    best_route = min(population, key=lambda r: route_distance(r, cities))
    best_distance = route_distance(best_route, cities)

    print("Starting Genetic Algorithm...\n")

```

```

no_improvement_count = 0

for gen in range(1, generations + 1):
    new_population = []
    for _ in range(pop_size):
        parent1 = selection(population, cities)
        parent2 = selection(population, cities)
        child = crossover(parent1, parent2)
        child = mutate(child)
        new_population.append(child)

    population = new_population
    current_best = min(population, key=lambda r: route_distance(r, cities))
    current_distance = route_distance(current_best, cities)

    if current_distance < best_distance:
        best_route, best_distance = current_best, current_distance
        no_improvement_count = 0
    else:
        no_improvement_count += 1

    print(f"Generation {gen}: Best Distance = {best_distance:.2f}")

    # Stop if no improvement for 3 consecutive generations
    if no_improvement_count >= 3:
        print("\nNo improvement for 3 generations. Stopping early.")
        break

return best_route, best_distance

# Example run
if __name__ == "__main__":
    city_count = 10
    cities = [(random.randint(0, 100), random.randint(0, 100)) for _ in
range(city_count)]

    print(f"Number of Cities: {city_count}")
    print("City Coordinates:", cities, "\n")

    best_route, best_distance = genetic_algorithm(cities)

```

```
print("\nBest Route Found:", best_route)
print("Best Distance:", best_distance)
```

Output:

```
Output Clear
Number of Cities: 10
City Coordinates: [(16, 1), (65, 96), (30, 42), (58, 100), (40, 67),
                  (70, 25), (67, 7), (64, 64), (49, 56), (16, 18)]

Initializing cities and population...
Starting Genetic Algorithm...

Generation 1: Best Distance = 305.39
Generation 2: Best Distance = 305.39
Generation 3: Best Distance = 279.97
Generation 4: Best Distance = 279.97
Generation 5: Best Distance = 279.97
Generation 6: Best Distance = 279.97

No improvement for 3 generations. Stopping early.

Best Route Found: [9, 0, 6, 5, 7, 1, 3, 8, 4, 2]
Best Distance: 279.97113971002153

=== Code Execution Successful ===
```

PROGRAM 2

Gene expression algorithm – Financial modelling and forecasting

Pseudocode

Input: Historical Financial data (prices, volumes, indicators)

Output: Best mathematical expression to predict future values

1. Initialize population of random expressions (genes)
2. For each generation
 - a. Translate genes into mathematical expressions.
 - b. Evaluate fitness = prediction error (e.g. MSE).
 - c. Select best individuals (tournament / roulette)
 - d. Apply crossover to combine expressions.
 - e. Apply mutation to introduce diversity
3. Keep track of best expression (lowest error)
4. Return the best evolved financial forecasting model

Output:

The algorithm will evolve mathematical expressions to predict stock prices.

Generation 1: Best Expr = $(x + 10) * 5$, Error = 34.12

Generation 2: Best Expr = $(x * 3) + 5$, Error = 6.45

Generation 3: Best Expr = $(x * 3) + 5$, Error = 6.45

Generation 4: Best Expr = $(x * 3) + 5$, Error = 6.45

No improvement for 3 consecutive generations, stopping early...

Final Best Expression: $(x * 3) + 5$

Code:

```
import random
```

```
import numpy as np
```

```

# -----
# 1. Create synthetic financial dataset
# -----

np.random.seed(42)

days = 40

prev_price = np.linspace(100, 140, days)      # yesterday's stock price
volume = np.random.randint(1000, 5000, days)   # trading volume
# True formula: price  $\approx$  0.8*prev_price + 0.01*volume + 5 + noise
y_data = 0.8 * prev_price + 0.01 * volume + 5 + np.random.normal(0, 2, days)

# -----
# 2. Define operators & terminals
# -----

OPERATORS = ['+', '-', '*']
TERMINALS = ['prev_price', 'volume', '5', '10']

def random_expr(depth=2):
    """Generate random expression tree as a string."""
    if depth == 0:
        return random.choice(TERMINALS)
    op = random.choice(OPERATORS)
    return f'({random_expr(depth-1)} {op} {random_expr(depth-1)})'

def eval_expr(expr, i):
    """Evaluate expression for a given day i."""
    try:
        return eval(expr, {"prev_price": prev_price[i], "volume": volume[i]})
    except:
        return float('inf')

```

```

def fitness(expr):
    """Mean Squared Error between predicted and actual values."""
    preds = [eval_expr(expr, i) for i in range(len(y_data))]
    if float('inf') in preds:
        return float('inf')
    return np.mean((np.array(preds) - y_data) ** 2)

def mutate(expr):
    """Mutation: randomly replace with new random expression."""
    return random_expr(depth=2) if random.random() < 0.3 else expr

def crossover(expr1, expr2):
    """Crossover: randomly choose one parent's expression."""
    return expr1 if random.random() < 0.5 else expr2

# -----
# 3. Gene Expression Algorithm (GEA)
# -----

def GEA(pop_size=20, generations=50, patience=3):
    print("Starting GEA for Financial Modeling...")
    print("Evolving formulas using features: prev_price and volume\n")

    # Initialize random population
    population = [random_expr(depth=3) for _ in range(pop_size)]
    best_expr, best_fit = None, float('inf')
    same_count = 0

    for gen in range(generations):
        # Compute fitness
        fitness_values = [fitness(expr) for expr in population]

```

```

gen_best_idx = np.argmin(fitness_values)

# Update best
if fitness_values[gen_best_idx] < best_fit:
    best_fit = fitness_values[gen_best_idx]
    best_expr = population[gen_best_idx]
    same_count = 0
else:
    same_count += 1

print(f'Gen {gen+1}: Best Expr = {best_expr}, Error = {best_fit:.2f}')

# Early stopping if no improvement
if same_count >= patience:
    print("\nNo improvement for 3 gens → stopping early.\n")
    break

# Tournament selection
selected = []
for _ in range(pop_size):
    a, b = random.sample(range(pop_size), 2)
    winner = population[a] if fitness_values[a] < fitness_values[b] else population[b]
    selected.append(winner)

# Generate offspring
offspring = []
for i in range(0, pop_size, 2):
    child1 = mutate(crossover(selected[i], selected[i+1]))
    child2 = mutate(crossover(selected[i+1], selected[i]))
    offspring.extend([child1, child2])

```

```

    population = offspring

    return best_expr, best_fit

# -----
# 4. Run GEA
# -----

best_expression, error = GEA()
print("Final Best Expression:", best_expression)
print("Final Prediction Error (MSE):", error)

```

OUTPUT:

Output

Clear

```

    prev_price) - (prev_price * 10))), Error = 17189.39
Gen 3: Best Expr = (((volume - volume) * (prev_price - volume)) * ((5 -
    prev_price) - (prev_price * 10))), Error = 17189.39
Gen 4: Best Expr = ((5 + prev_price) - (10 * 10)), Error = 11231.80
Gen 5: Best Expr = ((prev_price - prev_price) + (10 + prev_price)), Error
    = 149.79
Gen 6: Best Expr = ((prev_price - prev_price) + (10 + prev_price)), Error
    = 149.79
Gen 7: Best Expr = ((prev_price - prev_price) + (10 + prev_price)), Error
    = 149.79
Gen 8: Best Expr = ((prev_price - prev_price) + (10 + prev_price)), Error
    = 149.79

No improvement for 3 gens → stopping early.

Final Best Expression: ((prev_price - prev_price) + (10 + prev_price))
Final Prediction Error (MSE): 149.7940752928329

=== Code Execution Successful ===

```

PROGRAM 3

Particle swarm optimization – Robotics path planning

Pseudocode

Initialize parameters:

Success - rate, max-iterations

inertia-weight (w), cognitive-constant (c_1), social-constant (c_2)

Define environment:

start, goal, obstacles

Initialize each particle:

pos [i] = random path (sequence of waypoints from start to goal)

velocity [i] = random small changes in waypoints

personal-best [i] = pos [i]

personal-best-val [i] = fitness (pos [i])

find global-best from all personal-bests

for iter = 1 to max-iterations:

for each particle i :

Evaluate fitness (pos [i])

if fitness (pos [i]) < personal-best-val [i]:

personal-best [i] = pos [i]

personal-best-val [i] = fitness (pos [i])

update global-best among all personal bests

for each particle i :

for each waypoint j in path:

velocity [i][j] = $w * \text{velocity} [i][j] + c_1 * \text{rand}() * (\text{personal-best} [i][j] - \text{pos} [i][j])$

$$+ c2 \cdot \text{rand}() \cdot (\text{global_best}[j] - \text{pos}[i][j])$$

$$\text{position}[i][j] = \text{position}[i][j] + \text{velocity}[i][j]$$

Repair path if it collides with obstacle

if global-best-value has not improved for some iterations:

STOP

& if global-best-val $< 1e-5 \rightarrow$ break.

Return global-best optimal path from start to goal

Output:

PSO

Goal: Find the shortest path to the target point (5,5)

Fitness function: Euclidean distance to goal. Lower is better

Initialization:

- Number of particles: 30

- Iterations (max): 100

- Inertia (w) = 0.7, Cognitive ($c1$): 1.5, Social ($c2$): 1.5

- Search space bounds: (-10, 10)

Iteration 10/100 $\rightarrow$ Best Distance: 0.0455

Iteration 20/100 $\rightarrow$ Best Distance: 0.0207

Iteration 30/100 $\rightarrow$ Best Distance: 0.00447

Iteration 40/100 $\rightarrow$ Best Distance: 0.000821

Iteration 50/100 $\rightarrow$ Best Distance: 0.000186

Iteration 60/100 $\rightarrow$ Best Distance: 0.00000623

Iteration 70/100 $\rightarrow$ Best Distance: 0.0000182

Iteration 79/100 $\rightarrow$ Best Distance: 0.0000028299

Stopped early at iteration 76 (converged)

CODE:

```
import random
import math

# --- Particle Swarm Optimization for Robotic Path Planning ---
print("=== Particle Swarm Optimization (PSO) ===")
print("Goal: Find the shortest path to the target point (5, 5).")
print("Fitness Function: Euclidean distance to goal. Lower is better.")
print("Initialization:")
print("- Number of particles: 30")
print("- Iterations (max): 100")
print("- Inertia (w): 0.7, Cognitive (c1): 1.5, Social (c2): 1.5")
print("- Search space bounds: (-10, 10)")
print("=====\\n")

# Target point
goal = [5, 5]

# Parameters
num_particles = 30
num_iterations = 100
w = 0.7    # inertia
c1 = 1.5   # cognitive
c2 = 1.5   # social
bounds = (-10, 10)

# Fitness function = Euclidean distance
def fitness(position):
    return math.sqrt((position[0] - goal[0])**2 + (position[1] - goal[1])**2)
```

```

# Initialize particles
particles = []
velocities = []
personal_best = []
personal_best_scores = []

for _ in range(num_particles):
    pos = [random.uniform(bounds[0], bounds[1]), random.uniform(bounds[0], bounds[1])]
    vel = [random.uniform(-1, 1), random.uniform(-1, 1)]
    particles.append(pos)
    velocities.append(vel)
    personal_best.append(pos[:])
    personal_best_scores.append(fitness(pos))

# Global best
global_best = min(personal_best, key=fitness)
global_best_score = fitness(global_best)

# Iterations
for t in range(num_iterations):
    for i in range(num_particles):
        # Update velocity
        r1, r2 = random.random(), random.random()
        velocities[i][0] = (w * velocities[i][0] +
                           c1 * r1 * (personal_best[i][0] - particles[i][0]) +
                           c2 * r2 * (global_best[0] - particles[i][0]))
        velocities[i][1] = (w * velocities[i][1] +
                           c1 * r1 * (personal_best[i][1] - particles[i][1]) +
                           c2 * r2 * (global_best[1] - particles[i][1]))

```

```

# Update position
particles[i][0] += velocities[i][0]
particles[i][1] += velocities[i][1]

# Clamp within bounds
particles[i][0] = max(bounds[0], min(bounds[1], particles[i][0]))
particles[i][1] = max(bounds[0], min(bounds[1], particles[i][1]))

# Evaluate fitness
score = fitness(particles[i])
if score < personal_best_scores[i]:
    personal_best[i] = particles[i][:]
    personal_best_scores[i] = score

# Update global best
if score < global_best_score:
    global_best = particles[i][:]
    global_best_score = score

# Print progress
if (t+1) % 10 == 0 or global_best_score < 1e-5:
    print(f"Iteration {t+1}/{num_iterations} -> Best Distance: {global_best_score:.10f}")

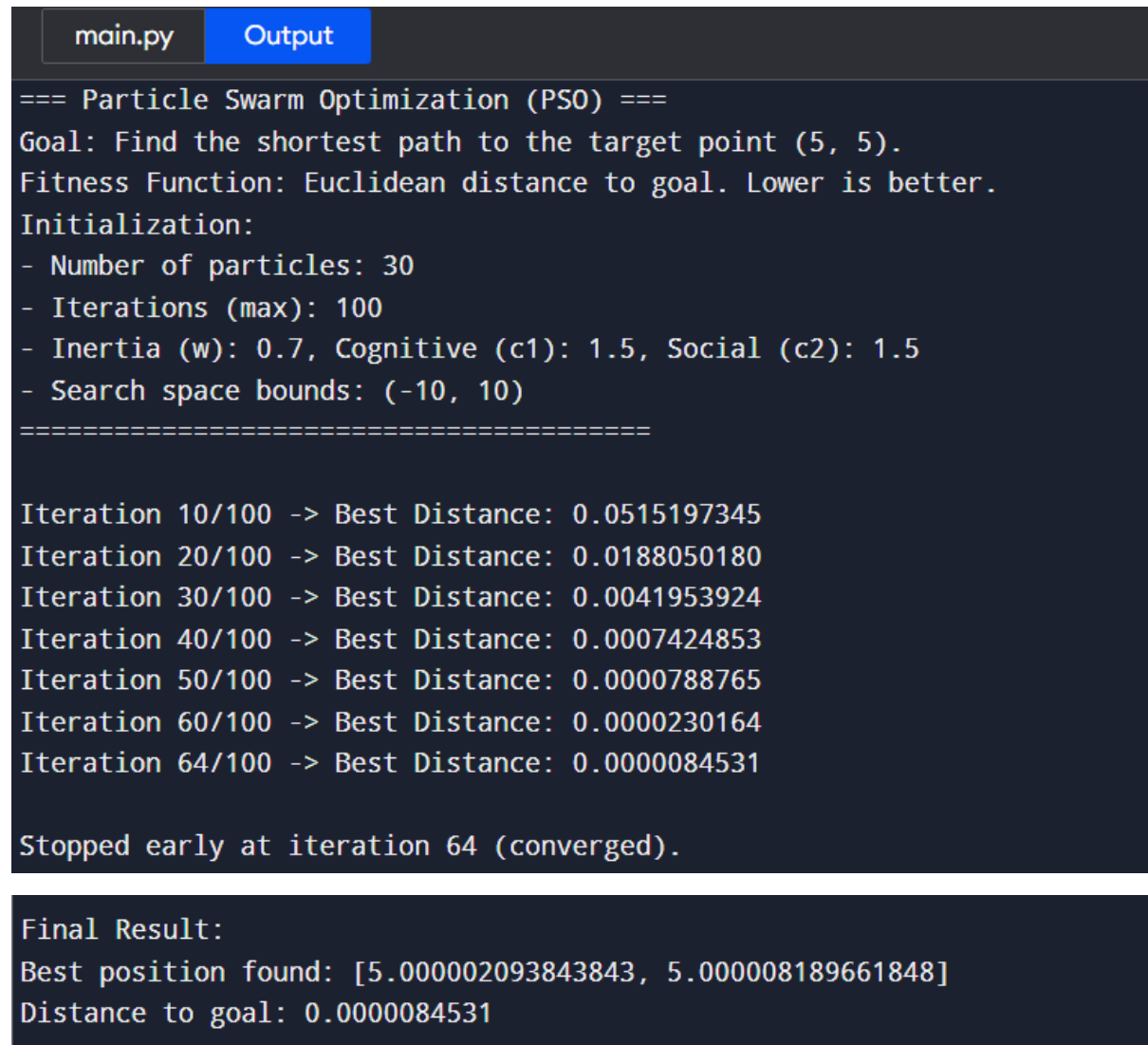
# Early stopping if close enough
if global_best_score < 1e-5:
    print(f"\nStopped early at iteration {t+1} (converged).")
    break

# Final result

```

```
print("\nFinal Result:")  
print(f"Best position found: {global_best}")  
print(f"Distance to goal: {global_best_score:.10f}")
```

OUTPUT:



```
main.py  Output  
=== Particle Swarm Optimization (PSO) ===  
Goal: Find the shortest path to the target point (5, 5).  
Fitness Function: Euclidean distance to goal. Lower is better.  
Initialization:  
- Number of particles: 30  
- Iterations (max): 100  
- Inertia (w): 0.7, Cognitive (c1): 1.5, Social (c2): 1.5  
- Search space bounds: (-10, 10)  
=====
```

Iteration	Best Distance
10/100	0.0515197345
20/100	0.0188050180
30/100	0.0041953924
40/100	0.0007424853
50/100	0.0000788765
60/100	0.0000230164
64/100	0.0000084531

```
Stopped early at iteration 64 (converged).  
  
Final Result:  
Best position found: [5.000002093843843, 5.000008189661848]  
Distance to goal: 0.0000084531
```

PROGRAM 4

Ant colony optimization – Job shop scheduling for manufacturing

Lab-4

8/9/25

Ant Colony Optimization

The foraging behavior of ants has inspired the development of optimization algorithms that can solve complex problems such as TSP. Ant colony optimization simulates the way ants find the shortest path to food sources & their nest. Implement ACO using python.

Job Shop Scheduling for manufacturing

JSS optimizes the assignment of tasks to machines in manufacturing minimizing makespan (total completion time) or tardiness. It's critical in industries like automobiles or electronics where efficient scheduling reduces production costs.

ACO's ability to handle complex constraints & its success in benchmarks make it a robust choice as noted in recent reviews on scheduling applications.

Problem: Schedule n jobs on m machines, each job with a specific operations sequence & processing times to minimize makespan. (total time)

Pseudocode:

1. Initialize parameters:

- num ants, num-iterations
- alpha (pheromone importance), beta (heuristic importance)
- rho (pheromone evaporation rate), Q (pheromone deposit factor)
- stop-count-threshold (number of unchanged iteration before stopping)

2. Read job data

- Each job has a sequence of operations
- Each operation has a machine & processing time.

3. Initialization

- pheromone matrix with small positive value for all job operations
- heuristic matrix as $1/\text{processing time}$ for all job operations
- best schedule - null
- best makespan - infinity
- unchanged_counters = 0
- previous best makespan - ∞ .

4. for iteration = 1 to num_iter:

a. for each ant from 1 to num_ants

i. Start with empty schedule

ii. For each job, set next operation index to 0

iii. While all jobs are not completed:

- Find eligible jobs that can be scheduled next

- For each eligible job

* compute probability based on pheromone & heuristic:

$$\text{probability} \propto (\text{pheromone}^\alpha) * (\text{heuristic}^\beta)$$

- select a job probabilistically

- Add selected operation to schedule

- Move to the next operation for that job

- If job completed, mark it done

iv. Calculate the makespan for the schedule considering machine & job constraints

v. Store schedule & makespan
 vi. If makespan < best makespan :
 * update best-schedule & best-makespan
 b. update pheromone table:
 - For each job-operation, evaporate pheromone;
 $\text{pheromone} \leftarrow (1 - \rho) * \text{pheromone}$
 - For each ant's schedule
 * For each operation in schedule
 $\text{pheromone} += \frac{1}{\text{makespan}}$
 c. Check for unchanged best solution
 - If current best-makespan = prev-best makespan
 unchanged-counter + 1
 - Else
 unchanged-counter = 0
 prev-best-makespan = best-makespan
 d. If ~~unchanged-counter~~ $\geq$ stop-count-threshold
 ~~stop the algorithm early & output the best schedule~~
 5. Output the final best schedule & the makespan

CODE:

```
import numpy as np
```

```
import random
```

```
# -----
```

```
# PARAMETERS INITIALIZATION
```

```
# -----
```

```
num_ants = 10          # Number of ants constructing solutions
```

```
num_iterations = 100    # Maximum number of iterations to run
```

```
alpha = 1.0            # Importance of pheromone
```

```
beta = 2.0              # Importance of heuristic (processing time)
```

```
rho = 0.5               # Evaporation rate of pheromone
```

```

Q = 1.0                # Amount of pheromone deposited based on solution quality

stop_count_threshold = 4    # Stop if the best solution doesn't improve for these many
iterations

# -----
# PROBLEM DATA
# -----

jobs_data = [
    [(0, 3), (1, 2), (2, 2)], # Job 0
    [(0, 2), (2, 1), (1, 4)], # Job 1
    [(1, 4), (2, 3), (0, 1)] # Job 2
]
num_jobs = len(jobs_data)
num_machines = 3

# -----
# INITIAL MATRICES
# -----

pheromone = np.ones((num_jobs, len(jobs_data[0]))) # Pheromone matrix initialized equally
heuristic = np.zeros((num_jobs, len(jobs_data[0]))) # Heuristic matrix based on processing
time

for j in range(num_jobs):
    for o in range(len(jobs_data[j])):
        heuristic[j][o] = 1.0 / jobs_data[j][o][1] # Heuristic = 1 / processing time

# -----
# TRACKING BEST SOLUTION
# -----

```

```

best_schedule = None
best_makespan = float('inf')
unchanged_counter = 0
prev_best_makespan = float('inf')

# -----
# PRINT INITIALIZED VALUES
# -----

print("===== INITIALIZED PARAMETERS =====")
print(f"Number of Ants      : {num_ants}")
print(f"Max Iterations      : {num_iterations}")
print(f"Alpha (pheromone)    : {alpha}")
print(f"Beta (heuristic)     : {beta}")
print(f"Rho (evaporation)    : {rho}")
print(f"Q (pheromone deposit): {Q}")
print(f"Stop count threshold : {stop_count_threshold}\n")

print("===== JOB DATA =====")
for job_id, operations in enumerate(jobs_data):
    print(f"Job {job_id}: {operations}")
print()

print("===== INITIAL MATRICES =====")
print("Pheromone Matrix:")
print(pheromone)
print("\nHeuristic Matrix:")
print(heuristic)
print("\nInitialization complete.\n")

```

```

# -----
# INTRODUCTION
# -----

print("This program applies Ant Colony Optimization to solve the Job Shop Scheduling
problem.")

print("Ants build schedules using pheromone trails and heuristic information to find the
schedule with the minimum makespan.")

print("The algorithm stops if the best solution has not improved for 4 consecutive
iterations.\n")

# -----
# FUNCTION DEFINITIONS
# -----

def construct_schedule():
    next_op = [0 for _ in range(num_jobs)]
    completed_jobs = 0
    schedule = []

    while completed_jobs < num_jobs:
        eligible_jobs = []
        for j in range(num_jobs):
            if next_op[j] < len(jobs_data[j]):
                eligible_jobs.append(j)

        probs = []
        for j in eligible_jobs:
            op_idx = next_op[j]
            tau = pheromone[j][op_idx] ** alpha

```

```

    eta = heuristic[j][op_idx] ** beta
    probs.append(tau * eta)

total = sum(probs)
probs = [p / total for p in probs]

selected_job = random.choices(eligible_jobs, weights=probs, k=1)[0]
selected_op = next_op[selected_job]

schedule.append((selected_job, selected_op))
next_op[selected_job] += 1

if next_op[selected_job] == len(jobs_data[selected_job]):
    completed_jobs += 1

return schedule

def calculate_makespan(schedule):
    machine_times = [0 for _ in range(num_machines)]
    job_times = [0 for _ in range(num_jobs)]

    for job_id, op_idx in schedule:
        machine_id, proc_time = jobs_data[job_id][op_idx]
        start_time = max(machine_times[machine_id], job_times[job_id])
        end_time = start_time + proc_time

        machine_times[machine_id] = end_time
        job_times[job_id] = end_time

    return max(machine_times)

```

```

def update_pheromones(ant_schedules, ant_makespans):
    global pheromone
    pheromone = (1 - rho) * pheromone
    for schedule, makespan in zip(ant_schedules, ant_makespans):
        deposit = Q / makespan
        for job_id, op_idx in schedule:
            pheromone[job_id][op_idx] += deposit

# -----
# MAIN ACO LOOP
# -----

for iteration in range(1, num_iterations + 1):
    ant_schedules = []
    ant_makespans = []

    for ant in range(num_ants):
        schedule = construct_schedule()
        makespan = calculate_makespan(schedule)

        ant_schedules.append(schedule)
        ant_makespans.append(makespan)

        if makespan < best_makespan:
            best_makespan = makespan
            best_schedule = schedule

    update_pheromones(ant_schedules, ant_makespans)

```

```

print(f'Iteration {iteration}: Best Makespan = {best_makespan}')

if best_makespan == prev_best_makespan:
    unchanged_counter += 1
else:
    unchanged_counter = 0
    prev_best_makespan = best_makespan

if unchanged_counter >= stop_count_threshold:
    print(f'\nBest solution unchanged for {stop_count_threshold} consecutive iterations.')
    print("Stopping early.\n")
    break

# -----
# OUTPUT RESULTS
# -----

print("===== FINAL RESULT =====")
print("Best Schedule Found:")
for job_id, op_idx in best_schedule:
    machine_id, proc_time = jobs_data[job_id][op_idx]
    print(f'Job {job_id}, Operation {op_idx} on Machine {machine_id} for {proc_time}
units")

print(f'\nOptimal Makespan: {best_makespan}')

```

OUTPUT:

```
===== INITIALIZED PARAMETERS =====
```

```
Number of Ants      : 10
Max Iterations      : 100
Alpha (pheromone)   : 1.0
Beta (heuristic)    : 2.0
Rho (evaporation)   : 0.5
Q (pheromone deposit): 1.0
Stop count threshold : 4
```

```
===== JOB DATA =====
```

```
Job 0: [(0, 3), (1, 2), (2, 2)]
Job 1: [(0, 2), (2, 1), (1, 4)]
Job 2: [(1, 4), (2, 3), (0, 1)]
```

```
===== INITIAL MATRICES =====
```

```
Pheromone Matrix:
```

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

```
Heuristic Matrix:
```

```
[[0.33333333 0.5      0.5      ]
 [0.5        1.        0.25     ]
 [0.25       0.33333333 1.       ]]
```

```
Initialization complete.
```

```
This program applies Ant Colony Optimization to solve the Job Shop
Scheduling problem.
```

```
Ants build schedules using pheromone trails and heuristic information to
find the schedule with the minimum makespan.
```

```
The algorithm stops if the best solution has not improved for 4
consecutive iterations.
```

```
Iteration 1: Best Makespan = 12
Iteration 2: Best Makespan = 12
Iteration 3: Best Makespan = 11
Iteration 4: Best Makespan = 11
Iteration 5: Best Makespan = 11
```

Iteration 7: Best Makespan = 11

Best solution unchanged for 4 consecutive iterations.
Stopping early.

===== FINAL RESULT =====

Best Schedule Found:

Job 1, Operation 0 on Machine 0 for 2 units
Job 1, Operation 1 on Machine 2 for 1 units
Job 2, Operation 0 on Machine 1 for 4 units
Job 0, Operation 0 on Machine 0 for 3 units
Job 2, Operation 1 on Machine 2 for 3 units
Job 0, Operation 1 on Machine 1 for 2 units
Job 2, Operation 2 on Machine 0 for 1 units
Job 0, Operation 2 on Machine 2 for 2 units
Job 1, Operation 2 on Machine 1 for 4 units

Optimal Makespan: 11

PROGRAM 5

Cuckoo Search – TSP

Lab-5.

Cuckoo Search (CS)

Cuckoo Search (CS) is a nature inspired optimization algorithm based on the brood parasitism of some cuckoo species. Their behaviour involves laying eggs in the nests of other birds, leading to the optimization of survival strategies. CS uses Levy flights to generate new solutions, promoting global search capabilities & avoiding local minima. The algorithm is widely used for solving continuous optimization problems & has applications in various domains, including engineering design, machine learning & data mining.

Application:

Traveling Salesman problem:

Pseudocode:

Initialize parameters:

population_size = n

max_iterations = $maxT$

discovery_rate = P_a

Initialize host nests with random tours (path visiting all cities)

while $t < maxT$ do

for each cuckoo i in population:

Generate new solution using Levy flight

Evaluate fitness (total tour length) F_i

Choose a random nest j among n
Evaluate fitness F_j of nest j

If $F < F_j$ then

Replace nest j with cuckoo i 's solution

End If

End For

Abandon a fraction P_a of worst nests:

Sort nests by fitness

Replace the worst P_a fraction with new random solutions

Keep the best solution found so far

Pass the best solution to the next generation

$t = t + 1$

End while

Print the best tour & the total distance.

Output:

Cuckoo Search Algorithm for the Traveling Salesman Problem (TSP)

We initialize a population of random tours & iteratively improve them using Levy flights & replacing the worst tours.

CODE:

```
import numpy as np
```

```
import random
```

```

# -----

# Display initializations and explanation
print("Cuckoo Search Algorithm for the Traveling Salesman Problem (TSP)")

print("We initialize a population of random tours and iteratively improve them using Levy
flights and replacing the worst tours.\n")

# -----

# Parameters
n = 10          # Number of nests (population size)
max_iter = 100  # Maximum number of iterations
Pa = 0.25       # Discovery rate of worst nests

# Example cities (coordinates)
cities = np.array([
    [0, 0],
    [1, 5],
    [5, 2],
    [6, 6],
    [8, 3],
    [2, 1],
    [7, 7],
    [3, 6],
    [4, 4],
    [9, 5]
])

num_cities = len(cities)

# Print initialized parameters
print("Parameters initialized:")
print(f"- Number of nests (population size): {n}")

```

```

print(f'- Maximum iterations: {max_iter}")
print(f'- Discovery rate of worst nests (Pa): {Pa}")
print(f'- Number of cities: {num_cities}")
print(f'- City coordinates:")
for i in range(num_cities):
    print(f' City {i}: {cities[i].tolist()}")
print()

# -----
# Function to calculate the total distance of a tour
def total_distance(tour):
    dist = 0
    for i in range(len(tour)):
        start = cities[tour[i % num_cities]]
        end = cities[tour[(i + 1) % num_cities]]
        dist += np.linalg.norm(start - end)
    return dist

# Generate initial population of random tours
nests = []
for _ in range(n):
    tour = list(range(num_cities))
    random.shuffle(tour)
    nests.append(tour)

# Evaluate fitness of each nest
def fitness(tour):
    return total_distance(tour)

# Levy flight function (simplified as random swap for TSP)

```

```

def levy_flight(tour):
    new_tour = tour.copy()
    i, j = random.sample(range(num_cities), 2)
    new_tour[i], new_tour[j] = new_tour[j], new_tour[i]
    return new_tour

# Main loop
best_tour = None
best_distance = float('inf')

for iteration in range(max_iter):
    for i in range(n):
        new_tour = levy_flight(nests[i])
        new_distance = fitness(new_tour)

        if new_distance < fitness(nests[i]):
            nests[i] = new_tour

# Abandon worst nests
nests = sorted(nests, key=lambda tour: fitness(tour))
num_abandon = int(Pa * n)
for i in range(n - num_abandon, n):
    tour = list(range(num_cities))
    random.shuffle(tour)
    nests[i] = tour

# Update best solution
current_best = nests[0]
current_distance = fitness(current_best)
if current_distance < best_distance:

```

```

    best_distance = current_distance

    best_tour = current_best

# Print progress every 10 iterations
if iteration % 10 == 0:
    print(f'Iteration {iteration}: Best distance = {best_distance:.2f}')

# Final output
print("\nBest tour found:")
print(best_tour)
print(f'Total distance: {best_distance:.2f}')

```

OUTPUT:

```

Cuckoo Search Algorithm for the Traveling Salesman Problem (TSP)
We initialize a population of random tours and iteratively improve them using Levy
    flights and replacing the worst tours.

Parameters initialized:
- Number of nests (population size): 10
- Maximum iterations: 100
- Discovery rate of worst nests (Pa): 0.25
- Number of cities: 10
- City coordinates:
    City 0: [0, 0]
    City 1: [1, 5]
    City 2: [5, 2]
    City 3: [6, 6]
    City 4: [8, 3]
    City 5: [2, 1]
    City 6: [7, 7]
    City 7: [3, 6]
    City 8: [4, 4]

```

City 8: [4, 4]

City 9: [9, 5]

Iteration 0: Best distance = 45.31

Iteration 10: Best distance = 31.73

Iteration 20: Best distance = 31.73

Iteration 30: Best distance = 29.13

Iteration 40: Best distance = 29.09

Iteration 50: Best distance = 29.09

Iteration 60: Best distance = 28.54

Iteration 70: Best distance = 28.54

Iteration 80: Best distance = 28.54

Iteration 90: Best distance = 28.54

Best tour found:

[0, 1, 8, 7, 3, 6, 9, 4, 2, 5]

Total distance: 28.54

PROGRAM 6

Grey wolf optimizer – Image segmentation

Pseudocode:

Input:

- Image I (grayscale)
- Number of wolves N
- Number of iterations T
- Search space bounds: $lb = 0, ub = 255$

Output:

- Best threshold value(s)
- Segmented Image

1. Initialize positions of N wolves randomly within $[lb, ub]$
2. Initialize α, β, δ wolves & their fitness scores, $a = 2$ decreases linearly.
3. For $t = 1$ to T
For each wolf in population:
a. Evaluate fitness using Otsu's between-class variance:

control factor - a
controls exploration

- plot image pixels based on wolf's threshold (3)
 - Compute probability & mean of each group
 - Fitness = negative of between-class variance
 b. Update α, β, δ values if current fitness is better
 c. Update position of each wolf:
 for each dimension j (threshold value):
 - Compute new position using position of α, β, δ :
 $A1 = 2 * a * r1 - a$ $C1 = 2 * r2$ $r1, r2 \sim \text{random}(0,1)$
 $D\text{-alpha} = |C1 * \text{Alpha}[j] - \text{Wolf}[i][j]|$ (distance from α)
 $X1 = \text{Alpha}[j] - A1 * D\text{-alpha}$ (new distance influenced by d)
 (Analogously for Beta $\rightarrow X2$, Delta $\rightarrow X3$)
 A - Step control factor
 how far wolf moves toward/away from leader
 $r1, r2 \sim \text{pr}$ $r1, r2 \sim \text{dist}$
 D - Distance wolf factor
 + adds stochasticity by adding distance
 + makes movement unpredictable
 Update $\text{Wolf}[i][j] = (X1 + X2 + X3) / 3$
 - clip $\text{Wolf}[i][j]$ within $[lb, ub]$
 d. Decrease control parameter 'a' linearly from 2 to 0
 4. End for
 5. Return Alpha wolf's position as Best Threshold
 6. Segment the image using Best Threshold:
 for each pixel p in image:
 If $p < \text{Best Threshold} \rightarrow \text{assign } 0 \text{ (dark)}$
 Else $\rightarrow \text{assign } 255 \text{ (bright)}$
 7. Output segmented image.

CODE:

import random

=====

Grey Wolf Optimizer

```

# =====

def gwo(obj_func, dim, lb, ub, wolves=10, iterations=20):
    positions = [[random.uniform(lb, ub) for _ in range(dim)] for _ in range(wolves)]
    alpha_pos, beta_pos, delta_pos = [0]*dim, [0]*dim, [0]*dim
    alpha_score, beta_score, delta_score = float("inf"), float("inf"), float("inf")

    for t in range(iterations):
        for i in range(wolves):
            fitness = obj_func(positions[i])
            if fitness < alpha_score:
                alpha_score = fitness; alpha_pos = positions[i][:]
            elif fitness < beta_score:
                beta_score = fitness; beta_pos = positions[i][:]
            elif fitness < delta_score:
                delta_score = fitness; delta_pos = positions[i][:]

        a = 2 - t * (2 / iterations)
        for i in range(wolves):
            for j in range(dim):
                r1, r2 = random.random(), random.random()
                A1, C1 = 2*a*r1 - a, 2*r2
                D_alpha = abs(C1*alpha_pos[j] - positions[i][j])
                X1 = alpha_pos[j] - A1*D_alpha

                r1, r2 = random.random(), random.random()
                A2, C2 = 2*a*r1 - a, 2*r2
                D_beta = abs(C2*beta_pos[j] - positions[i][j])
                X2 = beta_pos[j] - A2*D_beta

                r1, r2 = random.random(), random.random()

```

```

A3, C3 = 2*a*r1 - a, 2*r2
D_delta = abs(C3*delta_pos[j] - positions[i][j])
X3 = delta_pos[j] - A3*D_delta

positions[i][j] = min(max((X1+X2+X3)/3, lb), ub)

return sorted(int(x) for x in alpha_pos)

# =====
# Fitness Function (simple thresholding)
# =====
def otsu_fitness(thresholds, image):
    t = int(thresholds[0])
    group1 = [p for row in image for p in row if p < t]
    group2 = [p for row in image for p in row if p >= t]
    if not group1 or not group2: # avoid empty group
        return float("inf")
    mean1, mean2 = sum(group1)/len(group1), sum(group2)/len(group2)
    return -(len(group1)*mean1**2 + len(group2)*mean2**2)

# =====
# Main Program
# =====
# Fake 6x6 grayscale image (two regions: dark and bright)
image = [
    [50,50,50,200,200,200],
    [50,50,50,200,200,200],
    [50,50,50,200,200,200],
    [50,50,50,200,200,200],
    [50,50,50,200,200,200],

```

```

[50,50,50,200,200,200],
]

# Run GWO to find best threshold
best_threshold = gwo(lambda x: otsu_fitness(x, image),
                     dim=1, lb=0, ub=255, wolves=8, iterations=15)

print("Best Threshold found by GWO:", best_threshold)

# Apply segmentation
segmented = [[0 if p < best_threshold[0] else 255 for p in row] for row in image]
print("\nOriginal Image (pixel values):")
for row in image:
    print(row)
print("\nSegmented Image (0=dark, 255=bright):")
for row in segmented:
    print(row)

```

OUTPUT:

```

Best Threshold found by GWO: [114]

Original Image (pixel values):
[50, 50, 50, 200, 200, 200]
[50, 50, 50, 200, 200, 200]
[50, 50, 50, 200, 200, 200]
[50, 50, 50, 200, 200, 200]
[50, 50, 50, 200, 200, 200]
[50, 50, 50, 200, 200, 200]

Segmented Image (0=dark, 255=bright):
[0, 0, 0, 255, 255, 255]
[0, 0, 0, 255, 255, 255]
[0, 0, 0, 255, 255, 255]
[0, 0, 0, 255, 255, 255]
[0, 0, 0, 255, 255, 255]
[0, 0, 0, 255, 255, 255]

```

PROGRAM 7

Parallel cellular algorithm – Traffic flow simulation

7. Parallel Cellular Algorithm

13/10/25

Application: Traffic flow simulation

Traffic flow simulation models how vehicles move & interact on roads over time.

In this system, the road is divided into cells, where each cell can be either empty or occupied by a vehicle.

PCA are applied by updating all cells simultaneously based on the state of their neighbouring cells, following predefined movement rules.

This parallel & local update process effectively simulates vehicle movement, congestion & flow dynamics across the entire road network.

Algorithm:

1. Define problem:

- Road length = N
- number of cars = C
- Optimization goal: Maximize total traffic flow (cars moving)

2. Initialize Parameters:

- num_solutions = number of road configurations (cells)
- num_generations = number of time steps per simulation

3. Initialize Population:

- For each solution in population:
 - create a road array of length N
 - Randomly place C cars (1 = car, 0 = empty)

4. Evaluate Fitness:

- For each solution in population:

- total flow = 0

- Repeat num. iterations times:

- new-road = empty road

- moved = 0

- For each cell in current road:

- If cell has car:

- If next cell is empty:

- move car to next cell in new-road

- moved += 1

- Else:

- Keep car in current cell in new-road

- Update road = new-road

- total-flow += moved

- fitness of solution = total-flow

5. Find Best solution:

- best-solution = solution with maximum fitness

- best-fitness = corresponding total-flow

6. Output:

- Display all solutions with their fitness

- Display best-solution with best fitness

Code:

```
import random
```

```
# Parameters
```

```
road_length = 20
```

```
num_cars = 8
```

```
num_iterations = 10
```

```
num_solutions = 5 # Number of different road configurations (cells in PCA sense)
```

```
def generate_random_road():
```

```
    """Generate a random initial road configuration."""
```

```
    road = [0] * road_length
```

```
    car_positions = random.sample(range(road_length), num_cars)
```

```
    for pos in car_positions:
```

```
        road[pos] = 1
```

```
    return road
```

```
def simulate_traffic(road):
```

```
    """Simulate traffic for one configuration and calculate fitness (flow)."""
```

```
    total_flow = 0
```

```
    for _ in range(num_iterations):
```

```
        new_road = [0] * road_length
```

```
        moved = 0
```

```
        for i in range(road_length):
```

```
            if road[i] == 1:
```

```
                next_pos = (i + 1) % road_length
```

```
                if road[next_pos] == 0:
```

```
                    new_road[next_pos] = 1
```

```
                    moved += 1
```

```
            else:
```

```
                new_road[i] = 1
```

```

    road = new_road
    total_flow += moved
return total_flow # higher flow = better

# Step 1: Initialize population (multiple road configurations)
population = [generate_random_road() for _ in range(num_solutions)]

# Step 2: Evaluate fitness for each configuration
fitness_values = [simulate_traffic(road[:]) for road in population]

# Step 3: Find best solution
best_index = fitness_values.index(max(fitness_values))
best_road = population[best_index]
best_fitness = fitness_values[best_index]

# Step 4: Display results
print("Traffic Flow Optimization using PCA\n")
for i, (road, fit) in enumerate(zip(population, fitness_values)):
    print(f"Solution {i+1}: {' '.join(['🚗' if x else '.' for x in road])} | Flow = {fit}")

print("\n✅ Best Solution:")
print(" ".join(['🚗' if x else '.' for x in best_road]))
print(f"Maximum Traffic Flow = {best_fitness}")

```

Output

Traffic Flow Optimization using PCA

Solution 1: 🚗🚗...🚗🚗🚗🚗🚗...🚗 | Flow = 66

Solution 2: ..🚗🚗...🚗...🚗🚗...🚗🚗...🚗 | Flow = 79

Solution 3: ..🚗🚗🚗...🚗...🚗...🚗...🚗🚗 | Flow = 76

Solution 4: ...🚗🚗🚗...🚗...🚗🚗🚗...🚗... | Flow = 73

Solution 5: ...🚗...🚗🚗🚗...🚗...🚗🚗...🚗... | Flow = 74

✅ Best Solution:

..🚗🚗...🚗...🚗...🚗...🚗...🚗

Maximum Traffic Flow = 79

=== Code Execution Successful ===